



Arquivos

:::: Introdução a Algoritmos

Porque usar arquivos?

1. Crie uma lista simples de notas de alunos no seu programa e execute-o.
2. Feche a IDE (ou o programa) e abra-o novamente.

Onde estão os dados da lista? Foram perdidos!

O Problema da Volatilidade

Persistência de Dados

Variáveis e coleções operam na **memória principal (RAM)**, que é volátil e temporária.

Para que os dados sobrevivam após o computador ser desligado, precisamos enviá-los para a **memória secundária (HD/SSD)**.

Fazemos isso através da manipulação de **arquivos**.

Abrindo e fechando arquivos

A função **open()** retorna um objeto de arquivo.

Para fechar um arquivo, basta chamar **close()** explicitamente.

```
1 arquivo = open("dados.txt", "w")
2 arquivo.write("Olá, mundo!")
3 arquivo.close() # deve ser chamado sempre
```

Cria um arquivo **dados.txt**

👉 Se o programa der erro antes do `close()`, o arquivo corrompe ou fica "travado" no sistema operacional. O Risco da Abordagem Tradicional

Abrindo e fechando arquivos

A função **open()** retorna um objeto de arquivo.

Para fechar um arquivo, basta chamar **close()** explicitamente.

```
1 arquivo = open("dados.txt", "w")
2 arquivo.write("Olá, mundo!")
3 arquivo.close() # deve ser chamado
```

Escreve “*Olá, mundo!*” no
arquivo

👉 Se o programa der erro antes do `close()`, o arquivo corrompe ou fica "travado" no sistema operacional. O Risco da Abordagem Tradicional

Abrindo e fechando arquivos

A função **open()** retorna um objeto de arquivo.

Para fechar um arquivo, basta chamar **close()** explicitamente.

```
1 arquivo = open("dados.txt", "w")
2 arquivo.write("Olá, mundo!")
3 arquivo.close() # deve ser chamado
```

fecha o arquivo e imediatamente libera qualquer recurso do sistema usado por ele.

👉 Se o programa der erro antes do `close()`, o arquivo corrompe ou fica "travado" no sistema operacional. O Risco da Abordagem Tradicional

Gerenciador de contexto with

```
1 with open("dados.txt", "w") as arq:  
2     | arq.write("Olá, mundo!\n")  
3 # aqui fora, o arquivo já está fechado automaticamente
```

O **with** é um gerenciador de contexto utilizado para simplificar a manipulação de recursos, garantindo que eles sejam abertos e fechados **automaticamente**.

dados.txt	
1	Olá, mundo!
2	

Experimentando com Modos

1. Execute o código anterior com a frase "**Tudo bem?**".

O que houve com o "Olá, Mundo!"?

Desafio: Use um laço **for** para adicionar o nome de 3 colegas.

👉 O modo **"w"** (Write) **apaga tudo** e começa do zero. O modo **"a"** (Append) abre o arquivo e **acrescenta** os novos dados ao final, sem apagar o conteúdo existente.

Modos de abertura de arquivos

Modo	Nome	O que faz	Arquivo existe?
"r"	Read	Leitura. Erro se o arquivo não existir.	Deve existir
"w"	Write	Escrita. Apaga o conteúdo anterior.	Cria se não existir
"a"	Append	Acrescenta ao final sem apagar.	Cria se não existir
"x"	Exclusive	Escrita. Erro se o arquivo já existir.	Não deve existir
"r+"	Read+Write	Leitura e escrita simultâneas.	Deve existir

Atenção: Padrão Importante

A função `.write()` não pula linha sozinha como o `print()`!

Sempre faça a concatenação manual do `"\n"` ao final da string se quiser que os dados fiquem em linhas diferentes.

```
1 with open("produtos.txt", "a") as arq:  
2     for i in range(5):  
3         arq.write(input("Nome do produto: ") + "\n")
```

O caractere `\n` é o escape para **quebra de linha** (new line).

Lendo Dados - Modo "r"

Para recuperar dados salvos, mudamos o modo para **"r"** (Read). Este é o **modo padrão**.

Atenção: Se o arquivo **não existir**, o Python gerará um erro **FileNotFoundError**.

Formas de Leitura

Lê tudo de uma vez:

```
conteudo = arq.read()
```

Lê o arquivo inteiro como uma única string. Ideal para arquivos **pequenos**.

Lê linha por linha:

```
for linha in arq:
```

Lê iterativamente. Forma mais eficiente para **bases de dados pesadas**.

Tratamento de Dados

Ao ler arquivos, o caractere `\n` (quebra de linha) vem "escondido" no final de cada string.

Limpeza e Organização

Removendo o `\n`:

`texto.strip()`

Remove espaços em branco e quebras de linha ocultas nas extremidades da string.

Separando Dados (CSV):

`dados.split(",")`

Transforma uma string estruturada em uma lista de strings baseada no delimitador.

Salvando Dicionários

Iterar com `.items()` nos permite salvar coleções inteiras formatando o texto de saída.

Exemplo de Escrita Estruturada

```
estoque = {"Caderno": 10, "Borracha": 25}
with open("estoque.txt", "w") as arq:
    for chave, valor in estoque.items():
        arq.write(f"{chave}, {valor}\n")
```

Prática com Arquivos CSV

A partir de agora, utilizaremos a base de dados **data.csv** para aplicar conceitos avançados de manipulação.

Leitura e Limpeza

- Modo **"r"** para leitura
- **strip()**: remove espaços e **\n**
- **split(',')**: separa colunas

Escrita de Dados

- Modo **"w"**: sobrepor arquivo
- Modo **"a"**: anexar ao final
- Persistência estruturada



Agora, é sua vez!

Utilize os conceitos acima para processar a base de dados e gerar relatórios customizados nos exercícios a seguir.



Agora, é sua vez!

Abra data.csv no modo de leitura. Leia linha por linha, limpe com `.strip()` e separe com `.split(",")`. Verifique se o ano da música é menor que 1925. Se a condição for verdadeira, grave o Nome da música e o Ano em um novo arquivo de texto chamado musicas_antigas.txt (modo de escrita).



Agora, é sua vez!

Crie um dicionário vazio chamado `contagem_anos`. Leia `data.csv` e isole o ano de cada linha. Se o ano já for uma chave no dicionário, incremente seu valor em 1. Se não, crie a chave valendo 1. Ao final (fora do bloco `with`), percorra com `.items()` e imprima o histograma no console.



Agora, é sua vez!

Abra o arquivo CSV para leitura e DOIS arquivos simultâneos para escrita: populares.txt e comuns.txt. Se o índice de popularidade for ≥ 50 , grave a linha inteira em populares.txt. Caso contrário, grave em comuns.txt. (Dica: Use with open(...) as arq1, open(...) as arq2:)



Agora, é sua vez!

Leia apenas as primeiras 20 linhas de data.csv. Crie um dicionário onde a chave é o ID da música e o valor é uma Tupla (Nome, Ano). Grave esse dicionário num arquivo extracao_manual.json, simulando no braço o formato JSON (adicione chaves { }, separador : e pulos de linha \n).



PUC Minas

© **PUC Minas** • Todos os direitos reservados, de acordo com o art. 184 do Código Penal e com a lei 9.610 de 19 de fevereiro de 1998.

Proibidas a reprodução, a distribuição, a difusão, a execução pública, a locação e quaisquer outras modalidades de utilização sem a devida autorização da Pontifícia Universidade Católica de Minas Gerais.